string. It reads arguments from the command line using `process.argv`, which you saw in chapter 2.

Listing 3.1. Counting words

```
text = process.argv[2]                          <-- Get the argument provided to the program.

if text
  words = text.split /,/                        <-- Count the partygoers.
  console.log "#{words.length} partygoers"
else
  console.log 'usage: coffee 3.1.coffee [text]'  <-- No string provided. Exit.
```

Use this program from the command line by typing `coffee` followed by the name of the file and then the arguments to the program:

```
coffee listing.3.1.coffee Batman,Joker,Wolverine,Sabertooth
```

What does this have to do with functions? Well, functions are a bit like miniprograms. Consider a function that counts comma-separated words in a string:

```
countWords = (text) -> text.split(/,/).length
# [Function]
```


Enter the one-line `countWords` function into your REPL and experiment:

```
countWords 'sight,smell,taste,touch,hearing'
# 5

countWords 'morning,noon,night'
# 3
```

Functions can do anything—think of the possibilities! Before writing more functions, though, you should review their different parts.

3.1.3. Anatomy

As you've seen, a function is written with parameters in parentheses on the left side of an arrow and the function body on the right. But not many